

School of Sciences
(Department of Mathematics)

APPLIED CALCULUS LAB
(26SOSP103)

MATLAB LABORATORY MANUAL



B.Tech. Programme (first sem.)
Academic Session: 2026-27

Name:

Roll No:

Group:

Instructor:

S. No.	List of Experiments	Date	Signature
1	Introduction to MATLAB: Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions		
2	Graphing functions and derivatives		
3	Successive differentiation and Leibnitz's Theorem of algebraic, trigonometric, exponential, and logarithmic functions		
4	Taylor and Maclaurin series expansion of one-variable functions and approximation of function values		
5	Graphical verification of Rolle's Theorem and Lagrange's Mean Value Theorem		
6	Homogenous functions, Euler's theorem and Multivariable Taylor series		
7	Determination and Visualization of Maxima and Minima of Functions of Two Variables		
8	Gradient Descent for Function Minimization		
9	Newton's method for Function Minimization		
10	Single and Multiple Linear Regression using Gradient Descent		



Experiment 1A : Introduction to MATLAB: Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions

1. Write an algorithm for Introduction to MATLAB (Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions).

Algorithm for Introduction to MATLAB (Basic commands, symbolic variables, arithmetic operations, and plotting of standard functions): To familiarize students with the MATLAB environment, variables, basic commands, arithmetic operations, symbolic variables, built-in mathematical functions, and graphical visualization of standard mathematical functions.

Input: Numerical values, symbolic variables, and mathematical expressions.

Output: Numerical results, symbolic computations, and graphical plots.

Step 1: Launch MATLAB and open a new script file or notebook.

Step 2: Explore the development environment, including the Command Window/Console, Workspace or Variable Explorer, Current Folder, Editor, and Command History.

Step 3: Create variables and assign numerical values.

Step 4: Perform arithmetic operations such as addition, subtraction, multiplication, division, modulus, and exponentiation.

Step 5: Evaluate built-in mathematical functions such as square root, exponential, logarithmic, and trigonometric functions.

Step 6: Define symbolic variables and create symbolic mathematical expressions.

Step 7: Perform symbolic computations such as simplification, expansion, and factorization.

Step 8: Generate a suitable range of values for the independent variable.

Step 9: Plot a standard mathematical function using appropriate plotting commands.

Step 10: Add a title, axis labels, legend (if required), and grid to the graph.

Step 11: Display the numerical results, symbolic results, and graphical output.

Step 12: Save the program and terminate the execution.

2. Create two variables $a = 15$ and $b = 5$. Perform the following operations:

- Addition
- Subtraction
- Multiplication
- Division
- Exponentiation
- Modulus

3. Evaluate the following mathematical expressions: $\sqrt{64}$, e^2 , $\sin\left(\frac{\pi}{6}\right)$, $\cos\left(\frac{\pi}{3}\right)$, $\log(100)$.
4. Define the symbolic expression $x^2 + 5x + 6$. Using MATLAB, Factorize the given expression.
5. Plot the following functions over the interval $0 \leq x \leq 2\pi$.
 - $y = \sin x$
 - $y = \cos x$
 Include the graph title, axis labels, and grid.
6. A company's monthly profit is modeled by $P(x) = x^2 + 4x + 5$, where x represents the number of months. Using MATLAB,
 - Define the function.
 - Evaluate the profit for $x = 5$
 - Plot the function for $0 \leq x \leq 10$.

%% Experiment 1A: Introduction to MATLAB

% Aim:

**% To perform basic arithmetic operations, symbolic computations,
% and plot a standard mathematical function.**

clc;

clear;

close all;

%% Step 3: Create variables and assign numerical values

a = 15;

b = 4;

%% Step 4: Perform arithmetic operations

addition = a + b;

subtraction = a - b;

multiplication = a * b;

division = a / b;

modulus = mod(a,b);

powerValue = a^b;

%% Step 5: Evaluate built-in mathematical functions

squareRoot = sqrt(a);

exponential = exp(2);

logarithm = log(a); **% Natural logarithm**

sineValue = sin(pi/6);

```
cosineValue = cos(pi/3);  
tangentValue = tan(pi/4);
```

%% Step 6: Define symbolic variables and expressions

```
syms x
```

```
f = x^2 + 5*x + 6;
```

%% Step 7: Perform symbolic computations

```
simplifiedExpr = simplify((x^2 - 1)/(x - 1));  
expandedExpr = expand((x + 2)^3);  
factoredExpr = factor(f);
```

%% Step 8: Generate values for the independent variable

```
xValues = -5:0.1:5;
```

%% Step 9: Plot a standard mathematical function

```
yValues = xValues.^2;
```

```
figure;  
plot(xValues, yValues, 'b', 'LineWidth', 2);
```

%% Step 10: Add title, labels, legend, and grid

```
title('Graph of  $y = x^2$ ');  
xlabel('x');  
ylabel('y');  
legend('y = x^2', 'Location', 'northwest');  
grid on;
```

%% Step 11: Display numerical and symbolic results

```
disp('=====');  
disp('    Numerical Computations');  
disp('=====');
```

```

fprintf('Addition      = %.2f\n', addition);
fprintf('Subtraction   = %.2f\n', subtraction);
fprintf('Multiplication = %.2f\n', multiplication);
fprintf('Division      = %.2f\n', division);
fprintf('Modulus       = %.2f\n', modulus);
fprintf('Exponentiation (a^b) = %.2f\n', powerValue);

fprintf('\nSquare Root of a    = %.4f\n', squareRoot);
fprintf('Exponential exp(2)    = %.4f\n', exponential);
fprintf('Natural Log of a      = %.4f\n', logarithm);
fprintf('sin(pi/6)           = %.4f\n', sineValue);
fprintf('cos(pi/3)           = %.4f\n', cosineValue);
fprintf('tan(pi/4)           = %.4f\n', tangentValue);

disp(' ');
disp('=====');
disp('    Symbolic Computations');
disp('=====');

disp('Original Expression:');
disp(f);

disp('Simplified Expression:');
disp(simplifiedExpr);

disp('Expanded Expression:');
disp(expandedExpr);

disp('Factored Expression:');
disp(factoredExpr);

disp(' ');
disp('Program executed successfully.');
```



Experiment 1B : Introduction to MATLAB: Decision Making and Looping Statements in MATLAB

1. Write an algorithm for Introduction to MATLAB (Decision Making and Looping Statements in MATLAB).

Algorithm for Introduction to MATLAB (Decision Making and Looping Statements in MATLAB): To familiarize students with conditional statements and iterative programming using MATLAB.

Input: Numerical values and logical conditions.

Output: Results obtained using conditional statements and looping constructs.

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the required variables.

Step 3: Read or assign input values.

Step 4: Implement conditional statements (if, if-else, nested if, or switch-case/match-case) to evaluate logical conditions.

Step 5: Implement a for loop to execute statements for a fixed number of iterations.

Step 6: Implement a while loop to execute statements until a specified condition becomes false.

Step 7: Use break and continue statements to control loop execution where required.

Step 8: Display the output.

Step 9: Save the program and terminate the execution.

2. Write a MATLAB program to determine whether a given number is positive, negative, or zero.

3. Write a MATLAB program to determine whether a given year is a leap year.

4. Write a MATLAB program to find the largest among three numbers using the **if-else** statement.

5. Using a **for** loop, compute the sum of the first 20 natural numbers.

6. Using a **while** loop, generate the multiplication table of a given number up to 10.

7. Using a **for** loop, generate and display the squares and cubes of the numbers from **1 to 10** in tabular form.

Solution:

```
%% Program 2: Check Positive, Negative or Zero  
clc;
```



```

clear;
% Step 2 & 3: Read input
num = input('Enter a number: ');
% Step 4: Decision Making using if-elseif-else
if num > 0
    fprintf('%g is a Positive Number.\n', num);
elseif num < 0
    fprintf('%g is a Negative Number.\n', num);
else
    fprintf('The number is Zero.\n');
end

```

%% Program 3: Leap Year

```

clc;
clear;
% Step 2 & 3: Read input
year = input('Enter a year: ');
% Step 4: Decision Making
if (mod(year,400)==0) || (mod(year,4)==0 && mod(year,100)~=0)
    fprintf('%d is a Leap Year.\n', year);
else
    fprintf('%d is Not a Leap Year.\n', year);
end

```

%% Program 4: Largest of Three Numbers

```

clc;
clear;
% Step 2 & 3: Read inputs
a = input('Enter first number: ');
b = input('Enter second number: ');
c = input('Enter third number: ');

% Step 4: Decision Making
if a >= b && a >= c
    largest = a;
elseif b >= a && b >= c
    largest = b;
else
    largest = c;
end
fprintf('Largest Number = %g\n', largest);

```

%% Program 5: Sum of First 20 Natural Numbers

```

clc;

```

```
clear;
```

% Step 5: For Loop

```
sumValue = 0;
```

```
for i = 1:20
```

```
    sumValue = sumValue + i;
```

```
end
```

% Step 8: Display Output

```
fprintf('Sum of first 20 natural numbers = %d\n', sumValue);
```

%% Program 6: Multiplication Table

```
clc;
```

```
clear;
```

% Step 2 & 3: Read input

```
num = input('Enter a number: ');
```

% Step 6: While Loop

```
i = 1;
```

```
fprintf('\nMultiplication Table of %d\n', num);
```

```
while i <= 10
```

```
    fprintf('%d x %d = %d\n', num, i, num*i);
```

```
    i = i + 1;
```

```
end
```

%% Program 7: Squares and Cubes of Numbers

```
clc;
```

```
clear;
```

```
% Step 5: For Loop
```

```
fprintf('-----\n');
```

```
fprintf(' Number\t Square\t Cube\n');
```

```
fprintf('-----\n');
```

```
for i = 1:10
```

```
    fprintf('%4d\t%6d\t%6d\n', i, i^2, i^3);
```

```
end
```

```
fprintf('-----\n');
```

%% Example: break Statement

```
clc;
```

```
clear;
```

```
for i = 1:20
```

```
    if i == 11
```

```
        break;
```

```
    end
```

```
    fprintf('%d ', i);  
end  
%% Example: continue Statement  
clc;  
clear;  
for i = 1:10  
    if mod(i,2)==0  
        continue;  
    end  
    fprintf('%d ', i);  
end
```



Experiment 2 : Graphing functions and derivatives

1. Write an algorithm for Graphing functions and derivatives.

Algorithm for Graphing functions and derivatives: To visualize the relationship between a function and its derivative by plotting both on the same coordinate system using MATLAB and to interpret the geometric meaning of the derivative.

Input: A differentiable function $f(x)$.

Output: Graphs of the function and its derivative, and graphical interpretation of their relationship.

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the independent variable x .

Step 3: Enter the function $f(x)$.

Step 4: Compute the first derivative $f'(x)$ symbolically or numerically.

Step 5: Generate a suitable range of values for x .

Step 6: Evaluate both $f(x)$ and $f'(x)$ over the specified interval.

Step 7: Plot the function $f(x)$ and its derivative $f'(x)$ on the same graph using different colors or line styles.

Step 8: Add an appropriate title, axis labels, legend, and grid to the graph.

Step 9: Observe the relationship between the function and its derivative by identifying intervals where the function is increasing, decreasing, or has stationary points.

Step 10: Display the graphical output and interpret the results.

Step 11: Save the program and terminate the execution.

2. Using MATLAB, plot $f(x) = x^3 - 3x$ and its first derivative on the same graph. Identify the stationary points of the function.

%% Experiment: Graph of a Function and its First Derivative

% Question 2

% Plot $f(x) = x^3 - 3x$ and its first derivative

clc;

clear;

close all;

%% Step 2: Define the independent variable

syms x

%% Step 3: Define the function

f = x^3 - 3*x;

```

%% Step 4: Compute the first derivative
df = diff(f, x);
%% Display the derivative
disp('The first derivative is:')
disp(df)
%% Step 5: Generate values of x
xValues = linspace(-3,3,500);
%% Step 6: Evaluate the function and its derivative
fValues = double(subs(f, x, xValues));
dfValues = double(subs(df, x, xValues));
%% Find stationary points
criticalPoints = double(solve(df == 0, x));
criticalValues = double(subs(f, x, criticalPoints));
%% Step 7: Plot the graphs
figure;
plot(xValues, fValues, 'b', 'LineWidth',2);
hold on;
plot(xValues, dfValues, 'r--', 'LineWidth',2);
plot(criticalPoints, criticalValues,...
     'ko','MarkerFaceColor','g','MarkerSize',8);

%% Step 8: Formatting
title('Function f(x)=x^3-3x and its First Derivative');
xlabel('x');
ylabel('y');
legend('f(x)=x^3-3x','f'(x)=3x^2-3','Stationary Points',...
     'Location','best');
grid on;

%% Step 9 & 10: Display results

disp('-----')
disp('Stationary Points')
disp('-----')

T = table(criticalPoints,criticalValues,...
     'VariableNames',{'x','f(x)'});

disp(T)

fprintf('\nInterpretation:\n');
fprintf('1. f'(x)>0 : Function is Increasing.\n');
fprintf('2. f'(x)<0 : Function is Decreasing.\n');
fprintf('3. f'(x)=0 : Stationary Points occur.\n');

```

Expected Output

Derivative
 $f'(x)=3x^2-3$
Stationary Points

x f(x)

-1 2

1 -2

Interpretation

- Increasing on $(-\infty, -1)$ and $(1, \infty)$
- Decreasing on $(-1, 1)$
- Local maximum at $(-1, 2)$
- Local minimum at $(1, -2)$

3. Using MATLAB, plot $f(x) = \sin x$ and its first derivative on the same graph. Compare the graphs and verify that the derivative of $\sin x$ is $\cos x$.

%% Experiment: Graph of $\sin(x)$ and its First Derivative

% Question 3

```
clc;
```

```
clear;
```

```
close all;
```

%% Step 2: Define the independent variable

```
syms x
```

%% Step 3: Define the function

```
f = sin(x);
```

%% Step 4: Compute the first derivative

```
df = diff(f,x);
```

%% Display derivative

```
disp('The first derivative is:')
```

```
disp(df)
```

%% Step 5: Generate x-values

```
xValues = linspace(-2*pi,2*pi,600);
```

%% Step 6: Evaluate the function and derivative

```
fValues = double(subs(f,x,xValues));
```

```
dfValues = double(subs(df,x,xValues));
```

%% Step 7: Plot both graphs

```
figure;
```

```
plot(xValues,fValues,'b','LineWidth',2);
```

```
hold on;
```

```
plot(xValues,dfValues,'r--','LineWidth',2);
```

%% Step 8: Formatting

```
title('Graphs of  $\sin(x)$  and its First Derivative');
```

```
xlabel('x');
```

```
ylabel('y');
```

```
legend('sin(x)', 'cos(x)', 'Location', 'best');
```

```
grid on;
```

%% Step 9 & 10: Interpretation

```
disp('-----')
```

```
disp('Verification')
```

```
disp('-----')
```

```
disp('Derivative of  $\sin(x)$  is:')
```

```
disp(df)
fprintf('\nObservation:\n');
fprintf('Whenever sin(x) is increasing, cos(x) is positive.\n');
fprintf('Whenever sin(x) is decreasing, cos(x) is negative.\n');
fprintf('The derivative becomes zero at the maxima and minima of sin(x).\n');
fprintf('Hence,  $d/dx(\sin(x)) = \cos(x)$  is verified graphically.\n');
```

Expected Output

Graph Interpretation

- The **blue curve** represents $\sin x$
- The **red dashed curve** represents $\cos x$.
- At the maxima and minima of $\sin x$, the derivative $\cos x$ is zero.
- When $\sin x$ is increasing, $\cos x > 0$.
- When $\sin x$ is decreasing, $\cos x < 0$.
- Thus, the graph confirms that



Experiment 3: Successive Differentiation

1. Write an algorithm for Successive Differentiation.

Algorithm for Successive Differentiation : To determine the successive derivatives of a given symbolic function $f(x)$ up to the n th order using MATLAB.

Input: Symbolic function $f(x)$, order of differentiation n .

Output: Successive derivatives of $f(x)$ up to the n th order.

Step 1: Define the symbolic variable x .

Step 2: Enter the symbolic function $f(x)$.

Step 3: Read the order of differentiation n .

Step 4: for $i = 1 : n$

$d(i) = \text{diff}(f, x, i);$

Display $d(i);$

end

Step 5: Stop.

2. Write a program to determine the successive derivatives up to the fifth order of the function $f(x) = e^x \sin x$. Display all the derivatives and comment on the pattern observed in the successive derivatives.

%% Experiment: Successive Differentiation

% Function: $f(x) = \exp(x) \sin(x)$

clc;

clear;

close all;

%% Step 1: Define the symbolic variable

syms x

%% Step 2: Define the function

f = exp(x)*sin(x);

%% Step 3: Specify the order of differentiation


```

n = 5;

disp('-----')
disp('Successive Derivatives')
disp('-----')

disp('Given Function:')
disp(f)

%% Step 4: Compute successive derivatives

for i = 1:n

    d = simplify(diff(f, x, i));

    fprintf('\nDerivative of Order %d:\n', i);
    disp(d);

end

%% Step 5: End of Program
disp('-----')
disp('Program Executed Successfully')
disp('-----')

```

3. Write a MATLAB program to determine the first, second, and third order derivatives of the displacement function $f(t) = 3t^4 - 5t^3 + 2t^2 + 6t + 1$. Hence, determine the expressions for velocity, acceleration, and jerk, and discuss their physical significance.

```

%% Experiment: Successive Differentiation
% Motion Analysis using Derivatives

```

```

clc;
clear;
close all;

```

```

%% Step 1: Define the symbolic variable
syms t

```

```

%% Step 2: Define the displacement function

```

```

f = 3*t^4 - 5*t^3 + 2*t^2 + 6*t + 1;

```

```

%% Step 3: Specify the highest order of differentiation

```

```

n = 3;

disp('-----')
disp('Successive Derivatives of the Displacement')
disp('-----')

disp('Displacement Function:')
disp(f)

```

%% Step 4: Compute successive derivatives

```

for i = 1:n

    d(i) = diff(f, t, i);

    fprintf('\nDerivative of Order %d:\n', i);
    disp(d(i));

end

```

%% Velocity, Acceleration and Jerk

```

velocity = d(1);
acceleration = d(2);
jerk = d(3);

disp('-----')
disp('Physical Interpretation')
disp('-----')

disp('Velocity:')
disp(velocity)

disp('Acceleration:')
disp(acceleration)

disp('Jerk:')
disp(jerk)

disp('-----')
disp('Program Executed Successfully')
disp('-----')

```



Experiment 4: Taylor series expansion of one-variable functions

1. Write an algorithm for Function Approximation Using Taylor Series

Algorithm for Taylor series expansion of one-variable functions: To determine the approximate value of a given function using its Taylor series expansion.

Input : Symbolic function $f(x)$, expansion point a , approximation point x_0 , degree n

Output : Approximate value of $f(x_0)$

Step 1: Define the symbolic variable x .

Step 2: Enter the symbolic function $f(x)$.

Step 3: Read the values of a , x_0 and n .

Step 4: Compute the Taylor series expansion

$T = \text{taylor}(f, x, \text{'ExpansionPoint'}, a, \text{'Order'}, n+1);$

Step 5: Evaluate the Taylor polynomial at $x = x_0$.

Step 6: Display the approximate value.

Step 7: Stop.

2. Write a MATLAB program to determine the Taylor series expansion (Maclaurin series about $x = 0$) of the function $f(x) = e^x$ up to the sixth-degree term. Display the polynomial, plot both the original function and its approximation over the interval $[-2, 2]$, and comment on the accuracy of the approximation.

```
%% Experiment: Taylor Series Expansion (Maclaurin Series)
% Question 2
% Maclaurin series of f(x)=exp(x) up to the 6th-degree term
clc;
clear;
close all;
%% Step 1: Define the symbolic variable
syms x
%% Step 2: Define the function
f = exp(x);
%% Step 3: Specify expansion point, approximation point and degree
a = 0;      % Expansion point (Maclaurin)
x0 = 1;     % Example approximation point
n = 6;      % Degree of Taylor polynomial
%% Step 4: Compute the Taylor series
T = taylor(f, x, 'ExpansionPoint', a, 'Order', n+1);
%% Step 5: Evaluate the Taylor polynomial at x = x0
approxValue = double(subs(T, x, x0));
```

```

exactValue = double(subs(f, x, x0));
%% Step 6: Display the results
disp('-----')
disp('Maclaurin Series of exp(x)')
disp('-----')
disp(T)
fprintf('\nApproximate value at x = %.2f = %.10f\n', x0, approxValue);
fprintf('Exact value      at x = %.2f = %.10f\n', x0, exactValue);
fprintf('Absolute Error = %.10e\n', abs(exactValue-approxValue));
%% Plot the function and Taylor approximation
xValues = linspace(-2,2,400);
fValues = exp(xValues);
Tfun = matlabFunction(T);
TValues = Tfun(xValues);

figure;
plot(xValues,fValues,'b','LineWidth',2);
hold on;
plot(xValues,TValues,'r--','LineWidth',2);
grid on;
title('Maclaurin Series Approximation of e^x');
xlabel('x');
ylabel('y');
legend('e^x','6th-Degree Taylor Polynomial','Location','northwest');

%% Comment
disp(' ')
disp('Observation:')
disp('The Taylor polynomial closely matches e^x near x = 0.')
disp('The approximation error increases as x moves away from the expansion point.')

```

3. The displacement of a vibrating particle is given by $f(t) = \sin t$. Write a MATLAB program to find the Taylor series expansion of the given function about $t = 0$ up to the 7th-degree term. Use the Taylor polynomial to calculate the displacement at $t = 0.5$. Compare the approximate result with the exact value and determine the approximation error.

```

%% Experiment: Taylor Series Expansion
% Question 3
% Taylor series of sin(t) up to the 7th-degree term

```

```

clc;
clear;
close all;

```

```

%% Step 1: Define the symbolic variable
syms t

```

```

%% Step 2: Define the function
f = sin(t);

```

```

%% Step 3: Specify expansion point, approximation point and degree
a = 0;      % Maclaurin series
t0 = 0.5;   % Approximation point
n = 7;      % Degree of Taylor polynomial
%% Step 4: Compute the Taylor series

```

```

T = taylor(f, t, 'ExpansionPoint', a, 'Order', n+1);
%% Step 5: Evaluate the Taylor polynomial
approxValue = double(subs(T,t,t0));
%% Exact value
exactValue = sin(t0);
%% Approximation error
error = abs(exactValue-approxValue);
%% Step 6: Display the results

disp('-----')
disp('Taylor Series of sin(t)')
disp('-----')

disp(T)

fprintf('\nApproximate displacement at t = %.2f = %.10f\n',t0,approxValue);

fprintf('Exact displacement      = %.10f\n',exactValue);

fprintf('Absolute Error          = %.10e\n',error);

%% Plot the function and Taylor approximation

tValues = linspace(-2*pi,2*pi,500);

fValues = sin(tValues);

Tfun = matlabFunction(T);
TVValues = Tfun(tValues);

figure;

plot(tValues,fValues,'b','LineWidth',2);
hold on;

plot(tValues,TVValues,'r--','LineWidth',2);

grid on;

title('Taylor Approximation of sin(t)');
xlabel('t');
ylabel('Displacement');

legend('sin(t)','7th-Degree Taylor Polynomial','Location','best');

%% Observation
disp(' ')
disp('Observation:')
disp('The Taylor polynomial accurately approximates sin(t) near t = 0.')
disp('The approximation error becomes larger as t moves farther from the origin.')
Expected Output
For t=0.5
Approximate displacement = 0.4794255332
Exact displacement      = 0.4794255386
Absolute Error          = 5.37 × 10^-9

```



Experiment 5A : Graphical Verification of Rolle's Theorem

1. Write an algorithm for Graphical verification of Rolle's Theorem and Lagrange's Mean Value Theorem.

Algorithm of Graphical Verification of Rolle's Theorem : To determine graphically whether a given function satisfies Rolle's Theorem on the interval $[a, b]$ using MATLAB.

Input : Symbolic function $f(x)$, interval $[a, b]$.

Output : Graph of the function and the value(s) of c

Step 1: Define the symbolic variable x .

Step 2: Enter the symbolic function $f(x)$.

Step 3: Specify the interval $[a, b]$

Step 4: Verify that

- (i) $f(x)$ is continuous on $[a, b]$,
- (ii) $f(x)$ is differentiable on (a, b) ,
- (iii) $f(a) = f(b)$

Step 5: Compute the first derivative $D = \text{diff}(f, x)$.

Step 6: Solve $D = 0$ to obtain the value(s) of c .

Step 7: Plot the graph of $f(x)$ over the interval $[a, b]$.

Step 8: Mark the end points and the point(s) c on the graph.

Step 9: Display the graph and verify that the tangent at c is parallel to the x -axis.

Step 10: Stop.

2. The height of a road above the ground is modeled by $h(x) = x^3 - 6x^2 + 9x + 1$, where x is the horizontal distance measured in kilometres. Using the MATLAB Symbolic Math Toolbox, verify Rolle's Theorem over the interval $[0, 3]$. Compute the point(s) at which the road has a horizontal tangent, and verify the theorem graphically by plotting the elevation curve.

%% Experiment: Graphical Verification of Rolle's Theorem

% Function: $h(x) = x^3 - 6x^2 + 9x + 1$

% Interval: $[0, 3]$

clc;

clear;

close all;

%% Step 1: Define the symbolic variable

syms x

%% Step 2: Define the function

```
f = x^3 - 6*x^2 + 9*x + 1;
```

```
%% Step 3: Specify the interval
```

```
a = 0;
```

```
b = 3;
```

```
%% Step 4: Verify the conditions of Rolle's Theorem
```

```
% Function values at the endpoints
```

```
fa = subs(f, x, a);
```

```
fb = subs(f, x, b);
```

```
disp('-----');
```

```
disp('Verification of Rolle's Theorem');
```

```
disp('-----');
```

```
fprintf('f(%d) = %.2f\n', a, double(fa));
```

```
fprintf('f(%d) = %.2f\n', b, double(fb));
```

```
% Check the endpoint condition
```

```
if isequal(simplify(fa - fb), sym(0))
```

```
    disp('Condition (iii): f(a) = f(b) is satisfied.');
```

```
else
```

```
    disp('Condition (iii): f(a) = f(b) is NOT satisfied.');
```

```
end
```

```
% Note:
```

```
disp('Condition (i): Polynomial functions are continuous on [a,b].');
```

```
disp('Condition (ii): Polynomial functions are differentiable on (a,b).');
```

```
%% Step 5: Compute the first derivative
```

```
df = diff(f, x);
```

```
disp(' ');
```

```
disp('First Derivative:');
```

```
disp(df);
```

```
%% Step 6: Solve df = 0
```

```
criticalPoints = solve(df == 0, x);
```

```
% Select only points lying inside (a,b)
```

```
criticalPoints = double(criticalPoints);
```

```
c = criticalPoints(criticalPoints > a & criticalPoints < b);
```

```
fc = double(subs(f, x, c));
```

```
%% Display the values of c
```

```
disp(' ');
```

```
disp('Point(s) where f'(x)=0 :');
```

```

for i = 1:length(c)
    fprintf('c%d = %.4f\n', i, c(i));
end

```

%% Step 7: Plot the graph

```

xValues = linspace(a, b, 400);

fHandle = matlabFunction(f);

figure;

plot(xValues, fHandle(xValues), 'b', 'LineWidth', 2);
hold on;

```

%% Step 8: Mark the endpoints

```

plot(a, double(fa), 'ro', ...
     'MarkerFaceColor', 'r', 'MarkerSize', 8);

plot(b, double(fb), 'ro', ...
     'MarkerFaceColor', 'r', 'MarkerSize', 8);

```

%% Mark the Rolle's point(s)

```

plot(c, fc, 'ks', ...
     'MarkerFaceColor', 'g', 'MarkerSize', 9);

grid on;

title('Graphical Verification of Rolle's Theorem');
xlabel('x');
ylabel('h(x)');

legend('h(x)', ...
       'Endpoints', ...
       'Point(s) where h'(x)=0', ...
       'Location', 'best');

```

%% Step 9: Display conclusion

```

disp(' ');
disp('-----');

if ~isempty(c) && isequal(simplify(fa-fb), sym(0))
    disp('Rolle's Theorem is VERIFIED. ');
    fprintf('There exists at least one point c in (%.0f,%.0f)\n', a, b);
    fprintf('where h'(c)=0.\n');
    disp('The tangent at c is parallel to the x-axis. ');
end

```



```
else
    disp('Rolle's Theorem is NOT satisfied.');
```

```
end
```

Expected Output

Verification of Rolle's Theorem

$f(0) = 1.00$

$f(3) = 1.00$

Condition (i): Continuous ✓

Condition (ii): Differentiable ✓

Condition (iii): $f(0)=f(3)$ ✓

First Derivative

$3*x^2 - 12*x + 9$

Point(s) where $h'(x)=0$

$c1 = 1.0000$

Rolle's Theorem is VERIFIED.

There exists $c = 1$ in $(0,3)$

such that $h'(1)=0$.



Experiment 5B : Graphical verification of Lagrange's Mean Value Theorem

1. Write an algorithm for Graphical verification of Lagrange's Mean Value Theorem.

Algorithm of Graphical Verification of Rolle's Theorem : To determine graphically whether a given function satisfies Lagrange's Mean Value Theorem on the interval $[a, b]$ using MATLAB.

Input : Symbolic function $f(x)$, interval $[a, b]$

Output : Graph of the function and the value(s) of c

Step 1: Define the symbolic variable x .

Step 2: Enter the symbolic function $f(x)$.

Step 3: Specify the interval $[a, b]$.

Step 4: Verify that

- (i) $f(x)$ is continuous on $[a, b]$,
- (ii) $f(x)$ is differentiable on (a, b) .

Step 5: Compute the average slope $m = (f(b) - f(a))/(b - a)$.

Step 6: Compute the first derivative $D = \text{diff}(f, x)$.

Step 7: Solve $D = m$ to obtain the value(s) of c .

Step 8: Plot the graph of $f(x)$ over the interval $[a, b]$.

Step 9: Draw the secant line joining $(a, f(a))$ and $(b, f(b))$.

Step 10: Mark the point(s) c where the tangent is parallel to the secant line.

Step 11: Display the graph and verify the theorem graphically.

Step 12: Stop.

2. The motion of a vehicle is described by the displacement function $f(t) = t^3 - 3t^2 + 2t + 5$. Using MATLAB, verify Lagrange's Mean Value Theorem on the interval $[0, 2]$. Compute the average rate of change of displacement over the interval and determine the corresponding instant where the instantaneous rate of change is the same. Illustrate the result graphically.

```
%% Experiment: Graphical Verification of Lagrange's Mean Value Theorem
% Function:  $f(t) = t^3 - 3t^2 + 2t + 5$ 
% Interval:  $[0, 2]$ 
```

```
clc;
clear;
```

```

close all;

%% Step 1: Define the symbolic variable
syms t

%% Step 2: Define the function
f = t^3 - 3*t^2 + 2*t + 5;

%% Step 3: Specify the interval
a = 0;
b = 2;

%% Step 4: Verify the conditions of LMVT

fa = subs(f,t,a);
fb = subs(f,t,b);

disp('-----')
disp('Verification of Lagrange's Mean Value Theorem')
disp('-----')

fprintf('f(%d) = %.2f\n',a,double(fa));
fprintf('f(%d) = %.2f\n',b,double(fb));

disp('Condition (i): Polynomial functions are continuous on [a,b].');
disp('Condition (ii): Polynomial functions are differentiable on (a,b).');

%% Step 5: Compute the average slope

m = double((fb-fa)/(b-a));

fprintf('\nAverage Slope = %.4f\n',m);

%% Step 6: Compute the first derivative

df = diff(f,t);

disp(' ')
disp('First Derivative:')
disp(df)

%% Step 7: Solve f'(t) = Average Slope

criticalPoints = solve(df == m,t);

criticalPoints = double(criticalPoints);

% Select only points inside (a,b)
c = criticalPoints(criticalPoints>a & criticalPoints<b);

fc = double(subs(f,t,c));

%% Display the value(s) of c

disp(' ')
disp('Point(s) satisfying LMVT:')

for i=1:length(c)
    fprintf('c%d = %.4f\n',i,c(i));
end

```

```

%% Step 8: Plot the function

tValues = linspace(a,b,400);

fHandle = matlabFunction(f);

figure

plot(tValues,fHandle(tValues),'b','LineWidth',2)
hold on

%% Step 9: Draw the secant line

secant = m*(tValues-a) + double(fa);

plot(tValues,secant,'r--','LineWidth',2)

%% Step 10: Mark endpoints

plot(a,double(fa),'ko','MarkerFaceColor','k','MarkerSize',8)
plot(b,double(fb),'ko','MarkerFaceColor','k','MarkerSize',8)

%% Mark LMVT point(s)

plot(c,fc,'gs','MarkerFaceColor','g','MarkerSize',9)

grid on

title('Graphical Verification of Lagrange"s Mean Value Theorem')
xlabel('t')
ylabel('f(t)')

legend('f(t)',...
       'Secant Line',...
       'Endpoints',...
       'Point(s) c',...
       'Location','best')

%% Step 11: Display conclusion

disp(' ')
disp('-----')

if ~isempty(c)
    disp('Lagrange"s Mean Value Theorem is VERIFIED.')
    fprintf('Average slope = %.4f\n',m);

    for i=1:length(c)
        fprintf('f'(%0.4f) = %.4f\n',c(i),m);
    end

    disp('The tangent at c is parallel to the secant line.')
else
    disp('No point satisfying LMVT was found.')
end

```

Expected Output

Verification of Lagrange's Mean Value Theorem

$$f(0) = 5.00$$

$$f(2) = 1.00$$

$$\text{Average Slope} = -2.0000$$

First Derivative

$$3*t^2 - 6*t + 2$$

Point(s) satisfying LMVT

$$c1 = 1.5774$$

Lagrange's Mean Value Theorem is VERIFIED.

$$f(1.5774) = -2.0000$$

The tangent at c is parallel to the secant line.



Experiment 6 : Homogeneous Functions, Euler's Theorem, and Multivariable Taylor Series

1. Write an algorithm for Homogeneous Functions, Euler's Theorem, and Multivariable Taylor Series.

Algorithm of Homogeneous Functions, Euler's Theorem, and Multivariable Taylor Series :
To identify homogeneous functions, verify Euler's theorem, and compute the Taylor series expansion of multivariable functions using MATLAB.

Input: A multivariable function $f(x,y)$ and the point of expansion (a,b) .

Output: Identification of the homogeneous function, verification of Euler's theorem, and Taylor series approximation.

Part A: Verification of Euler's Theorem

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the symbolic variables x and y .

Step 3: Enter the given function $f(x,y)$.

Step 4: Determine whether the function is homogeneous and identify its degree n .

Step 5: Compute the first-order partial derivatives: f_x and f_y .

Step 6: Compute $L = x f_x + y f_y$.

Step 7: Compute $R = n f(x,y)$.

Step 8: Compare L and R . If $L = R$, conclude that Euler's theorem is verified.

Part B: Multivariable Taylor Series

Step 9: Enter the point (a,b) about which the Taylor series is to be expanded.

Step 10: Compute the required first- and second-order partial derivatives of $f(x,y)$.

Step 11: Construct the Taylor series expansion of the function about the point (a,b) .

Step 12: Display the Taylor polynomial obtained.

Step 13: Save the program and terminate the execution.

2. Using MATLAB, verify Euler's theorem for $f(x,y) = x^3 + 3x^2y + 3xy^2 + y^3$.

%% Experiment: Verification of Euler's Theorem

% Function: $f(x,y) = x^3 + 3x^2y + 3xy^2 + y^3$

```
clc;  
clear;  
close all;
```

%% Step 2: Define symbolic variables

```
syms x y
```

```

%% Step 3: Define the function
f = x^3 + 3*x^2*y + 3*x*y^2 + y^3;
%% Step 4: Identify the degree of the homogeneous function
n = 3;    % Since every term has total degree 3

```

```

disp('-----')
disp('Verification of Euler"s Theorem')
disp('-----')

```

```

fprintf('Degree of the homogeneous function = %d\n', n);

```

```

%% Step 5: Compute first-order partial derivatives

```

```

fx = diff(f, x);
fy = diff(f, y);

```

```

disp(' ')
disp('Partial derivative with respect to x:')
disp(fx)

```

```

disp('Partial derivative with respect to y:')
disp(fy)

```

```

%% Step 6: Compute L = x*fx + y*fy

```

```

L = simplify(x*fx + y*fy);

```

```

%% Step 7: Compute R = n*f

```

```

R = simplify(n*f);

```

```

%% Step 8: Compare L and R

```

```

disp(' ')
disp('Left-Hand Side (L) = x*fx + y*fy')
disp(L)

```

```

disp(' ')
disp('Right-Hand Side (R) = n*f(x,y)')
disp(R)

```

```

if simplify(L - R) == 0
    disp(' ')
    disp('Euler"s Theorem is VERIFIED.')
else
    disp(' ')
    disp('Euler"s Theorem is NOT VERIFIED.')
end

```

Expected Output

```

Degree of the homogeneous function = 3
fx = 3*x^2 + 6*x*y + 3*y^2
fy = 3*x^2 + 6*x*y + 3*y^2
L = 3*x^3 + 9*x^2*y + 9*x*y^2 + 3*y^3
R = 3*x^3 + 9*x^2*y + 9*x*y^2 + 3*y^3
Euler's Theorem is VERIFIED.

```

3. Using MATLAB, obtain the second-order Taylor series expansion of $f(x,y) = e^{x+y}$ about the point (0,0).

%% Experiment: Second-Order Taylor Series of a Function of Two Variables

% Function: $f(x,y) = \exp(x+y)$

clc;

clear;

close all;

%% Step 2: Define symbolic variables

syms x y

%% Step 3: Define the function

f = exp(x + y);

%% Step 9: Expansion point

a = 0;

b = 0;

%% Step 10: Compute first- and second-order partial derivatives

fx = diff(f, x);

fy = diff(f, y);

fxx = diff(f, x, 2);

fyy = diff(f, y, 2);

fxy = diff(f, x, y);

%% Evaluate derivatives at (0,0)

f0 = subs(f, [x y], [a b]);

fx0 = subs(fx, [x y], [a b]);

fy0 = subs(fy, [x y], [a b]);

fxx0 = subs(fxx, [x y], [a b]);

fyy0 = subs(fyy, [x y], [a b]);

fxy0 = subs(fxy, [x y], [a b]);

%% Step 11: Construct the second-order Taylor polynomial

T = f0 ...

+ fx0*(x-a) ...

+ fy0*(y-b) ...

+ (1/2)*(...

fxx0*(x-a)^2 ...

+ 2*fxy0*(x-a)*(y-b) ...

+ fyy0*(y-b)^2);


```
T = simplify(expand(T));
```

```
%% Step 12: Display the Taylor polynomial
```

```
disp('-----')
```

```
disp('Second-Order Taylor Polynomial')
```

```
disp('-----')
```

```
disp(T)
```

Expected Output

Second-Order Taylor Polynomial

$1 + x + y + \frac{x^2}{2} + x*y + \frac{y^2}{2}$



Experiment 7 : Determination and Visualization of Maxima and Minima of Functions of Two Variables

1. Write an algorithm for Determination and Visualization of Maxima and Minima of Functions of Two Variables

Algorithm for Determination and Visualization of Maxima and Minima of Functions of Two Variables: To determine and visualize the maxima and minima of functions of two variables using MATLAB by computing critical points, applying the second derivative test, and plotting the corresponding surface.

Input: A function $f(x,y)$.

Output: Critical points, nature of the critical points, and graphical visualization of the function.

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the symbolic variables x and y .

Step 3: Enter the given function $f(x,y)$.

Step 4: Compute the first-order partial derivatives f_x and f_y .

Step 5: Solve $f_x = 0$, $f_y = 0$ to obtain the critical points.

Step 6: Compute the second-order partial derivatives f_{xx} , f_{xy} , f_{yy} .

Step 7: Compute the Hessian determinant $D = f_{xx} f_{yy} - (f_{xy})^2$ at each critical point.

Step 8: Classify each critical point using the second derivative test:

- If $D > 0$ and $f_{xx} > 0$, the point is a **local minimum**.
- If $D > 0$ and $f_{xx} < 0$, the point is a **local maximum**.
- If $D < 0$, the point is a **saddle point**.
- If $D = 0$, the test is **inconclusive**.

Step 9: Generate a suitable mesh of values for x and y .

Step 10: Plot the function using a 3D surface plot and/or contour plot.

Step 11: Display the critical points, their nature, and the graphical visualization.

Step 12: Save the program and terminate the execution.

2. Using MATLAB, determine the critical points and identify the maxima or minima of $f(x,y) = x^2 + y^2$. Visualize the function using a surface plot.

%% Experiment: Maxima and Minima of a Function of Two Variables

% Function: $f(x,y) = x^2 + y^2$

```

clc;
clear;
close all;

%% Step 2: Define symbolic variables
syms x y

%% Step 3: Define the function
f = x^2 + y^2;

%% Step 4: Compute first-order partial derivatives
fx = diff(f,x);
fy = diff(f,y);

%% Step 5: Find the critical points
[xc,yc] = solve([fx==0, fy==0],[x,y]);

%% Step 6: Compute second-order partial derivatives
fxx = diff(fx,x);
fyy = diff(fy,y);
fxy = diff(fx,y);

disp('-----')
disp('Critical Point Analysis')
disp('-----')

for k = 1:length(xc)

    x0 = double(xc(k));
    y0 = double(yc(k));

    %% Step 7: Compute Hessian determinant
    D = double(subs(fxx*fyy - fxy^2,[x y],[x0 y0]));
    A = double(subs(fxx,[x y],[x0 y0]));

    fprintf('\nCritical Point : (%.2f , %.2f)\n',x0,y0);

    %% Step 8: Classification

    if D > 0 && A > 0
        nature = 'Local Minimum';
    elseif D > 0 && A < 0
        nature = 'Local Maximum';
    elseif D < 0
        nature = 'Saddle Point';
    else
        nature = 'Test Inconclusive';
    end

    fprintf('Nature : %s\n',nature);

end

%% Step 9: Generate mesh

```

```
[X,Y] = meshgrid(-4:0.1:4,-4:0.1:4);

Z = X.^2 + Y.^2;

%% Step 10: Surface plot

figure

surf(X,Y,Z)

hold on

plot3(x0,y0,double(subs(f,[x y],[x0 y0])),...
      'ro','MarkerFaceColor','r','MarkerSize',8)

xlabel('x')
ylabel('y')
zlabel('f(x,y)')

title('Surface Plot of f(x,y)=x^2+y^2')

grid on
```

Expected Output

Critical Point : (0.00,0.00)

Nature : Local Minimum

3. Using MATLAB determine the critical points of $f(x,y) = x^2 + y^2 - 4x - 6y$. Classify each critical point and plot the surface.

%% Experiment: Maxima and Minima of a Function of Two Variables

% Function: $f(x,y) = x^2 + y^2 - 4x - 6y$

```
clc;
clear;
close all;
```

%% Step 2: Define symbolic variables

```
syms x y
```

%% Step 3: Define the function

```
f = x^2 + y^2 - 4*x - 6*y;
```

%% Step 4: Compute first-order partial derivatives

```
fx = diff(f,x);
fy = diff(f,y);
```

%% Step 5: Find the critical points

```
[xc,yc] = solve([fx==0,fy==0],[x,y]);
```

%% Step 6: Compute second-order partial derivatives

```

fxx = diff(fx,x);
fyy = diff(fy,y);
fxy = diff(fx,y);

disp('-----')
disp('Critical Point Analysis')
disp('-----')

for k=1:length(xc)

    x0 = double(xc(k));
    y0 = double(yc(k));

    %% Step 7: Hessian Determinant

    D = double(subs(fxx*fyy-fxy^2,[x y],[x0 y0]));
    A = double(subs(fxx,[x y],[x0 y0]));

    fprintf('\nCritical Point : (%.2f , %.2f)\n',x0,y0);

    %% Step 8: Classification

    if D>0 && A>0
        nature='Local Minimum';
    elseif D>0 && A<0
        nature='Local Maximum';
    elseif D<0
        nature='Saddle Point';
    else
        nature='Test Inconclusive';
    end

    fprintf('Nature : %s\n',nature);

end

%% Step 9: Generate mesh

[X,Y]=meshgrid(-2:0.1:6,-2:0.1:8);

Z=X.^2+Y.^2-4*X-6*Y;

%% Step 10: Plot the surface

figure

surf(X,Y,Z)

hold on

plot3(x0,y0,double(subs(f,[x y],[x0 y0])),...
    'ro','MarkerFaceColor','r','MarkerSize',8)

xlabel('x')
ylabel('y')

```

```
zlabel('f(x,y)')
```

```
title('Surface Plot of  $f(x,y)=x^2+y^2-4x-6y$ ')
```

```
grid on
```

Expected Output

Critical Point : (2.00,3.00)

Nature : Local Minimum



Experiment 8 : Gradient Descent for Function Minimization

1. Write an algorithm for Gradient Descent for Function Minimization

Algorithm for Gradient Descent for Function Minimization: To implement the Gradient Descent algorithm to determine the minimum of a multivariable function using MATLAB.

Input: A differentiable function $f(x,y)$ an initial point (x_0, y_0) , learning rate α , tolerance ε , and maximum number of iterations.

Output: Approximate minimum point and minimum function value.

Step 1: Define the multivariable function $f(x,y)$.

Step 2: Compute the gradient components f_x, f_y

Step 3: Select an initial point (x_0, y_0) .

Step 4: Choose a suitable learning rate α , tolerance ε , and maximum number of iterations.

Step 5: Evaluate the gradient at the current point.

Step 6: Update the variables using

$$x_{k+1} = x_k - \alpha f_x$$

$$y_{k+1} = y_k - \alpha f_y$$

Step 7: Compute the new function value.

Step 8: Repeat Steps 5–7 until

- the gradient norm becomes smaller than the specified tolerance, **or**
- the maximum number of iterations is reached.

Step 9: Display the approximate minimum point and the corresponding minimum function value.

Step 10: Plot the optimization path on the contour or surface plot.

Step 11: Stop.

2. Minimize $f(x,y) = x^2 + 2y^2$. Plot the optimization path.

%% Experiment: Gradient Descent Optimization

% Question 2

% Minimize $f(x,y) = x^2 + 2y^2$

clc;

clear;

close all;

%% Step 1: Define the function

f = @(x,y) x.^2 + 2*y.^2;

```

%% Step 2: Gradient components
fx = @(x,y) 2*x;
fy = @(x,y) 4*y;

%% Step 3: Initial point
x = 4;
y = 3;

%% Step 4: Learning parameters
alpha = 0.10;    % Learning rate
tol = 1e-6;      % Tolerance
maxIter = 100;   % Maximum iterations

% Store optimization path
X = x;
Y = y;

%% Steps 5–8: Gradient Descent Iteration
for k = 1:maxIter

    % Compute gradient
    gx = fx(x,y);
    gy = fy(x,y);

    % Check stopping criterion
    if norm([gx gy]) < tol
        break;
    end

    % Update variables
    x = x - alpha*gx;
    y = y - alpha*gy;

    % Store points
    X(end+1) = x;
    Y(end+1) = y;

end

%% Step 9: Display result
fprintf('Approximate Minimum Point = (%.6f, %.6f)\n',x,y);
fprintf('Minimum Function Value = %.6f\n',f(x,y));
fprintf('Number of Iterations = %d\n',k);

%% Step 10: Plot optimization path

[xGrid,yGrid] = meshgrid(-5:0.1:5,-5:0.1:5);
z = f(xGrid,yGrid);

figure
contour(xGrid,yGrid,z,30)
hold on

plot(X,Y,'r-o','LineWidth',2,'MarkerSize',4)

```



```

plot(x,y,'kp','MarkerFaceColor','g','MarkerSize',12)

xlabel('x')
ylabel('y')
title('Gradient Descent Optimization Path')
legend('Contours','Optimization Path','Minimum','Location','best')

```

grid on

Expected Output

Approximate Minimum Point = (0.000000,0.000000)

Minimum Function Value = 0

Iterations $\approx$ 50

3. The loss function of a simple machine learning model is $F(x,y) = x^2 + y^2 - 6x - 4y + 20$. Use Gradient Descent to determine the optimal parameter values.

%% Experiment: Gradient Descent Optimization

% Question 3

% Minimize $F(x,y) = x^2 + y^2 - 6x - 4y + 20$

```

clc;
clear;
close all;

```

%% Step 1: Define the function

```
F = @(x,y) x.^2 + y.^2 - 6*x - 4*y + 20;
```

%% Step 2: Gradient components

```

Fx = @(x,y) 2*x - 6;
Fy = @(x,y) 2*y - 4;

```

%% Step 3: Initial point

```

x = 0;
y = 0;

```

%% Step 4: Learning parameters

```

alpha = 0.10;
tol = 1e-6;
maxIter = 100;

```

% Store optimization path

```

X = x;
Y = y;

```

%% Steps 5–8: Gradient Descent

```
for k = 1:maxIter
```

```

    gx = Fx(x,y);
    gy = Fy(x,y);

    if norm([gx gy]) < tol
        break;
    end

    x = x - alpha*gx;
    y = y - alpha*gy;

    X(end+1) = x;
    Y(end+1) = y;

end

%%% Step 9: Display result

fprintf('Optimal Point = (%.6f, %.6f)\n',x,y);

fprintf('Minimum Function Value = %.6f\n',F(x,y));

fprintf('Iterations = %d\n',k);

%%% Step 10: Contour plot

[xGrid,yGrid] = meshgrid(-1:0.1:6,-1:0.1:5);

Z = F(xGrid,yGrid);

figure

contour(xGrid,yGrid,Z,30)

hold on

plot(X,Y,'r-o','LineWidth',2,'MarkerSize',4)

plot(x,y,'kp','MarkerFaceColor','g','MarkerSize',12)

xlabel('x')

ylabel('y')

title('Gradient Descent for Machine Learning Loss Function')

legend('Contours','Optimization Path','Optimal Point','Location','best')

grid on

```

Expected Output

Optimal Point = (3.000000,2.000000)

Minimum Function Value = 7.000000

Iterations $\approx$ 60



Experiment 9 : Newton's method for Function Minimization

1. Write an algorithm for Newton's method for Function Minimization.

Algorithm for Newton's method for Function Minimization: To implement Newton's method for determining the minimum of a multivariable function using MATLAB.

Input: A twice differentiable function $f(x,y)$, an initial point (x_0, y_0) tolerance ϵ , and maximum number of iterations.

Output: Approximate minimum point and minimum function value.

Step 1: Launch MATLAB and create a new script file.

Step 2: Define the multivariable function $f(x,y)$.

Step 3: Compute the gradient vector $\nabla f(x,y)$

Step 4: Compute the Hessian matrix H

Step 5: Select an initial point (x_0, y_0) .

Step 6: Evaluate the gradient vector and Hessian matrix at the current point.

Step 7: Update the variables using Newton's iteration $x_{k+1} = x_k - H^{-1} \nabla f(x_k)$.

Step 8: Repeat Steps 6–7 until the norm of the gradient is less than the specified tolerance or the maximum number of iterations is reached.

Step 9: Display the approximate minimum point and the corresponding minimum function value.

Step 10: Plot the function surface or contour plot and indicate the minimum point.

Step 11: Save the program and terminate the execution.

2. Using MATLAB, determine the minimum of $f(x, y) = x^2 + y^2$, using Newton's method.

%% Experiment: Newton's Method for Optimization

% Question 2

% Minimize $f(x,y) = x^2 + y^2$

clc;

clear;

close all;

%% Step 2: Define symbolic variables and function

syms x y

f = x^2 + y^2;

%% Step 3: Compute the gradient vector

gradF = gradient(f,[x y]);

%% Step 4: Compute the Hessian matrix

```

H = hessian(f,[x y]);

%% Step 5: Select an initial point
X = [3; 4];

%% Parameters
tol = 1e-6;
maxIter = 20;

% Store iteration points
path = X;

%% Steps 6-8: Newton's Iteration

for k = 1:maxIter

    % Evaluate Gradient and Hessian
    G = double(subs(gradF,{x,y},{X(1),X(2)}));
    Hk = double(subs(H,{x,y},{X(1),X(2)}));

    % Stopping criterion
    if norm(G) < tol
        break;
    end

    % Newton update
    X = X - Hk\G;

    % Store path
    path(:,end+1) = X;

end

%% Step 9: Display Results

xmin = X(1);
ymin = X(2);

fmin = double(subs(f,{x,y},{xmin,ymin}));

fprintf('Minimum Point = (%.6f, %.6f)\n',xmin,ymin);
fprintf('Minimum Function Value = %.6f\n',fmin);
fprintf('Iterations = %d\n',k);

%% Step 10: Contour Plot

[Xg,Yg] = meshgrid(-5:0.1:5,-5:0.1:5);
Z = Xg.^2 + Yg.^2;

figure

contour(Xg,Yg,Z,30)
hold on

plot(path(1,:),path(2:,:), 'r-o', 'LineWidth', 2)

```

```

plot(xmin,ymin,'kp','MarkerFaceColor','g','MarkerSize',12)

xlabel('x')
ylabel('y')
title('Newton Method for  $f(x,y)=x^2+y^2$ ')

legend('Contours','Newton Path','Minimum','Location','best')
grid on

```

Expected Output

Minimum Point = (0.000000, 0.000000)

Minimum Function Value = 0.000000

3. Using MATLAB, minimize $f(x,y) = x^2 + 2y^2 - 8x - 12y + 30$. Determine the minimum point and the minimum function value.

```

%% Experiment: Newton's Method for Optimization

```

```

% Question 3

```

```

% Minimize  $f(x,y)=x^2+2y^2-8x-12y+30$ 

```

```

clc;
clear;
close all;

```

```

%% Step 2: Define symbolic variables and function

```

```

syms x y

```

```

f = x^2 + 2*y^2 - 8*x - 12*y + 30;

```

```

%% Step 3: Gradient vector

```

```

gradF = gradient(f,[x y]);

```

```

%% Step 4: Hessian matrix

```

```

H = hessian(f,[x y]);

```

```

%% Step 5: Initial point

```

```

X = [0;0];

```

```

%% Parameters

```

```

tol = 1e-6;
maxIter = 20;

```

```

path = X;

```

```
%% Steps 6-8: Newton Iteration
```

```
for k = 1:maxIter
```

```
    G = double(subs(gradF,{x,y},{X(1),X(2)}));
```

```
    Hk = double(subs(H,{x,y},{X(1),X(2)}));
```

```
    if norm(G) < tol
```

```
        break;
```

```
    end
```

```
    X = X - Hk\G;
```

```
    path(:,end+1) = X;
```

```
end
```

```
%% Step 9: Display Results
```

```
xmin = X(1);
```

```
ymin = X(2);
```

```
fmin = double(subs(f,{x,y},{xmin,ymin}));
```

```
fprintf('Minimum Point = (%.6f, %.6f)\n',xmin,ymin);
```

```
fprintf('Minimum Function Value = %.6f\n',fmin);
```

```
fprintf('Iterations = %d\n',k);
```

```
%% Step 10: Contour Plot
```

```
[Xg,Yg] = meshgrid(-1:0.1:8,-1:0.1:6);
```

```
Z = Xg.^2 + 2*Yg.^2 - 8*Xg - 12*Yg + 30;
```

```
figure
```

```
contour(Xg,Yg,Z,30)
```

```
hold on
```

```
plot(path(1,:),path(2,:),'r-o','LineWidth',2)
```

```
plot(xmin,ymin,'kp','MarkerFaceColor','g','MarkerSize',12)
```

```
xlabel('x')
```

```
ylabel('y')
```

```
title('Newton Method for Optimization')
```

```
legend('Contours','Newton Path','Minimum','Location','best')
```

```
grid on
```

Expected Output

Minimum Point = (4.000000, 3.000000)

Minimum Function Value = -4.000000



Experiment 10 : Single and Multiple Linear Regression using Gradient Descent

1. Write an algorithm for Single and Multiple Linear Regression using Gradient Descent

Algorithm for Single and Multiple Linear Regression using Gradient Descent : To implement single and multiple linear regression models using the Gradient Descent optimization algorithm and estimate the optimal model parameters using MATLAB.

Input: Training dataset, learning rate α , tolerance ε , and maximum number of iterations.

Output: Regression coefficients, fitted regression model, and predicted output values.

Step 1: Launch MATLAB and create a new script file.

Step 2: Enter or load the training dataset.

Step 3: Separate the independent variable(s) and the dependent variable.

Step 4: Initialize the regression parameters with suitable initial values.

Step 5: Choose the learning rate, tolerance, and maximum number of iterations.

Step 6: Compute the predicted output using the regression model.

Step 7: Evaluate the Mean Squared Error (MSE) cost function.

Step 8: Compute the gradient of the cost function with respect to each regression parameter.

Step 9: Update the regression parameters using the Gradient Descent update rule.

Step 10: Repeat Steps 6–9 until the convergence criterion is satisfied or the maximum number of iterations is reached.

Step 11: Display the optimal regression coefficients and the final cost.

Step 12: Plot the fitted regression line (for single regression) or compare the predicted and actual values (for multiple regression).

Step 13: Save the program and terminate the execution.

2. The following data represent the study hours and examination marks of five students.

Study Hours	Marks
1	40
2	50
3	60
4	70
5	80

Using MATLAB:

- Implement Single Linear Regression using Gradient Descent.
- Determine the regression equation.
- Plot the fitted regression line.
- Predict the marks of a student who studies 6 hours.

%% Experiment: Single Linear Regression Using Gradient Descent

```
clc;  
clear;  
close all;
```

%% Step 2: Enter the training dataset

```
X = [1; 2; 3; 4; 5];    % Study Hours  
Y = [40; 50; 60; 70; 80]; % Marks
```

%% Step 3: Separate the independent and dependent variables

```
m = length(X);          % Number of training samples
```

%% Step 4: Initialize regression parameters

```
theta0 = 0;              % Intercept  
theta1 = 0;              % Slope
```

%% Step 5: Learning parameters

```
alpha = 0.01;            % Learning rate  
tolerance = 1e-6;        % Stopping criterion  
maxIter = 5000;          % Maximum iterations
```

```
costHistory = zeros(maxIter,1);
```

%% Steps 6-10: Gradient Descent

```
for iter = 1:maxIter
```

% Step 6: Predicted output

```
Y_pred = theta0 + theta1*X;
```

% Step 7: Mean Squared Error (Cost Function)

```
cost = (1/(2*m))*sum((Y_pred - Y).^2);  
costHistory(iter) = cost;
```

% Step 8: Compute gradients

```
dtheta0 = (1/m)*sum(Y_pred - Y);  
dtheta1 = (1/m)*sum((Y_pred - Y).*X);
```

% Save previous parameters

```
old_theta0 = theta0;  
old_theta1 = theta1;
```

% Step 9: Update parameters

```
theta0 = theta0 - alpha*dtheta0;  
theta1 = theta1 - alpha*dtheta1;
```

% Step 10: Check convergence

```
if norm([theta0-old_theta0, theta1-old_theta1]) < tolerance  
    costHistory = costHistory(1:iter);  
    break;  
end
```

```
end
```

%% Step 11: Display the regression equation

```
fprintf('-----\n');  
fprintf('Single Linear Regression Using Gradient Descent\n');  
fprintf('-----\n');
```

```
fprintf('Intercept (theta0) = %.4f\n', theta0);  
fprintf('Slope (theta1)   = %.4f\n', theta1);
```

```
fprintf('\nRegression Equation:\n');  
fprintf('Marks = %.4f + %.4f × StudyHours\n', theta0, theta1);
```

```
fprintf('\nFinal Cost (MSE) = %.6f\n', cost);
```

%% Predict marks for 6 study hours

```
studyHours = 6;
```

```
predictedMarks = theta0 + theta1*studyHours;
```

```
fprintf('\nPredicted Marks for %.0f study hours = %.2f\n', ...  
        studyHours, predictedMarks);
```

%% Step 12: Plot the fitted regression line

```
figure
```

```
scatter(X,Y,80,'filled')  
hold on
```

```
xFit = linspace(min(X),6,100);  
yFit = theta0 + theta1*xFit;
```

```
plot(xFit,yFit,'r','LineWidth',2)
```

```
xlabel('Study Hours')  
ylabel('Marks')
```

```
title('Single Linear Regression using Gradient Descent')
```

```
legend('Training Data','Regression Line','Location','northwest')
```

```
grid on
```

%% Plot the Cost Function

```
figure
```

```
plot(1:length(costHistory),costHistory,'b','LineWidth',2)
```

```
xlabel('Iteration')  
ylabel('Cost (MSE)')
```

title('Convergence of Gradient Descent')

grid on

Single Linear Regression Using Gradient Descent

Expected Output

Intercept (theta0) = 30.0000

Slope (theta1) = 10.0000

Regression Equation

Marks = $30.0000 + 10.0000 \times \text{StudyHours}$

Final Cost (MSE) = 0.000000

Predicted Marks for 6 study hours = 90.00